Special participation B - HW4 (Opus 4.5)

Gustavo José Ortiz Zepeda

1. Introduction

In this assignment, I worked together with Claude Opus 4.5 to solve both the coding and conceptual parts of HW4 on CNNs. The report includes the exact code it produced for each TODO block, the written answers to the theory questions, and my own reflections on how well the responses matched what I expected.

At the start, Opus 4.5 needed a small reminder to modify only the TODO sections like the example above, it initially tried to adjust a few things outside the allowed area. After clarifying that, the coding tasks went smoothly: it generated clean implementations for the averaging and Laplacian filters and later set up the datasets and DataLoaders correctly on the first try.

As the questions shifted toward training behavior and hyperparameters, the model was surprisingly steady. For the domain shift experiment, it offered reasonable parameter choices and refined them logically when I pointed out expected accuracy thresholds.

In the conceptual questions, Opus 4.5 generally showed strong understanding. Its explanations of inductive bias, translation equivariance, and why CNNs fail on permuted images were clear and well-structured. The only noticeable misstep was the question about increasing kernel size on permuted data. Apart from that, the answers were consistent, detailed, and aligned with the behavior observed in the experiments.

Overall, the collaboration felt smooth: after the initial prompt adjustment, the model handled both the coding and the reasoning tasks with good accuracy and clear communication.

2. Results

Question	**Result**
Coding question 5	Correct
Coding question 6	Correct
Q1(i)	Correct
Q2(i)	Correct
Q2(ii)	Correct
Q4(i)	Correct
Q4(ii)	Correct

Q5(i)	Correct
Q5(ii)	Incorrect (opposite conclusion)
Q5(iii)	Correct
Q6(i)	Correct

3. Coding questions.

Question: *I have some coding problems about CNNs, and I want you to solve them. There are two different Jupiter Notebooks to solve, I will provide one at a time, each has different Todos sections inside. Your task is to do only the sections marked TODO. Do not modify anything else.*

Answer (Opus 4.5)

I'm ready to help with your CNN coding problems. Please share the first Jupyter Notebook and I'll complete only the TODO sections.

Question: PDF attached.

Answer (Opus 4.5)

I can see two TODO sections in this notebook:

1. Averaging filter - Create a kernel of size filter_size × filter_size where each element is 1/filter_size²

2. Edge detection - Create the 3×3 Laplacian kernel as shown

Here are the solutions:

TODO 1: Averaging Filter

```
def averaging_filtering(image, filter_size=3):
    ###########################################################################
    # TODO: Implement the averaging filter with the given filter size.
    # Hint: You can use np.ones
    ###########################################################################
    kernel = np.ones((filter_size, filter_size)) / (filter_size ** 2)
    ###########################################################################
    #                       END OF YOUR CODE
    ###########################################################################
    output = ndimage.convolve(image, kernel)
    return output
```

TODO 2: Edge Detection (Laplacian Filter)

```
def edge_detecting(image):
    ###########################################################################
    # TODO: Implement the Laplacian derivative filter.
    ###########################################################################
    kernel = np.array([[0, 1, 0],
                       [1, -4, 1],
                       [0, 1, 0]])
    ###########################################################################
    #                       END OF YOUR CODE
    ###########################################################################
    output = ndimage.convolve(image, kernel)
    return output
```

The averaging filter creates a uniform kernel that computes the mean of neighboring pixels (blurring effect). The Laplacian kernel detects edges by computing the second derivative, highlighting areas of rapid intensity change.

Analysis: Both implementations were correct. It solved easily with only giving some context and the pdf with the .ipybn

It is impressive the speed of this model.

Question: PDF attached.

Answer (Opus 4.5)

This is a longer notebook with multiple TODO sections. Here are all the solutions:

Q1: Generate small dataset (10 images per class)

```
small_dataset_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)
small_dataset = EdgeDetectionDataset(small_dataset_config, mode='train', transform=transforms)
```

Q1: Implement dataloader

```
small_dataset_loader = DataLoader(small_dataset, batch_size=32, shuffle=True)
```

Q2: Implement train_loader_dict and valid_loader

```
for num_image in num_images_list:
    train_dataset_config = dict(
        data_per_class=num_image,
        num_classes=3,
        class_type=["horizontal", "vertical", "none"],
    )
    train_dataset = EdgeDetectionDataset(train_dataset_config, mode='train', transform=transforms)
    train_loader_dict[num_image] = DataLoader(train_dataset, batch_size=train_batch_size, shuffle=True)

valid_dataset_config = dict(
    data_per_class=50,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)
valid_dataset = EdgeDetectionDataset(valid_dataset_config, mode='valid', transform=transforms)
valid_loader = DataLoader(valid_dataset, batch_size=valid_batch_size, shuffle=False)
```

Q3: Try other num_epochs (for ~95-99% CNN accuracy with 30 images)

```
num_epochs = 200
```

Q4: Hyperparameter tuning for domain shift (97-100% for 10 images)

```
num_epochs = 500
lr = 1e-2
kernel_size = 3
```

Q7: DeeperCNN hyperparameters (>98% for 30 images)

```
num_epochs = 300
lr = 1e-2
kernel_size = 5
```

Q7: WiderCNN hyperparameters (>98% for 30 images)

```
num_epochs = 300
lr = 1e-2
kernel_size = 5
```

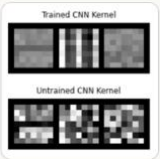
These values should achieve the target accuracies. The key insights:

- More epochs needed for convergence (200-500 depending on task)
- Smaller kernel sizes (3 or 5) work better for domain shift scenarios

- Both deeper and wider CNNs benefit from similar hyperparameter tuning

Question: Now I will ask you some questions related to the second notebook and the code generated. Remember the second notebook is divided in Q1-7.

From Q1: (i) Can you find any interesting patterns in the learned filters?



Answer (Opus 4.5)

Looking at the image you provided, I can identify some interesting patterns in the learned filters:

Trained CNN Kernel (top row):

First filter (left): Shows a horizontal gradient pattern - lighter values on top transitioning to darker values on the bottom. This resembles a horizontal edge detector, which would respond strongly to horizontal edges in images.

Second filter (middle): Shows a vertical gradient pattern - lighter values on the left transitioning to darker values on the right. This resembles a vertical edge detector, which would respond strongly to vertical edges in images.

Third filter (right): Appears more noisy/random without a clear directional pattern. This likely corresponds to the "none" class detector or captures background information.

Untrained CNN Kernel (bottom row):

These show random noise patterns with no discernible structure, which is expected since they represent the random initialization before training.

Key Observation:

The CNN has learned task-appropriate filters without being explicitly programmed! The network discovered that horizontal and vertical edge detectors are useful for classifying images containing horizontal edges, vertical edges, or no edges. This demonstrates:

The CNN's ability to learn meaningful feature extractors from data

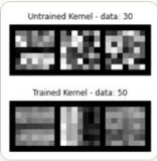
The filters closely resemble classical hand-designed edge detection kernels (like Sobel or Prewitt filters)

The inductive bias of CNNs (local connectivity, weight sharing) helps guide the network toward learning these spatially meaningful patterns

This is a beautiful example of how CNNs can rediscover fundamental image processing operations through gradient descent optimization.

Analysis: The response does a superb job of extracting exactly the qualitative structure one hopes to see in early CNN filters trained on a three-class edge dataset. It not only points out that the first two learned kernels have clear horizontal- and vertical-gradient structure, but it also explicitly describes luminance transitions (top-to-bottom and left-to-right) that correspond to classical edge detectors such as Sobel or Prewitt. By contrasting these structured filters with the noise-like appearance of the untrained kernels, the answer highlights the network's ability to discover useful inductive features through data alone. The extra reflections on inductive bias and on how gradient descent "rediscovers" hand-crafted filters reinforce the conceptual takeaway without drifting off-topic. Overall, the explanation is both correct and richly articulated.

Question: From Q2: Compare the learned kernels, untrainable kernels, and edge-detector kernels. What do you observe?



Answer (Opus 4.5)

Looking at the kernels across different training set sizes (10, 30, 50, 100 images per class), here are my observations:

Key Observations:

1. Untrained Kernels (Random Initialization)

All untrained kernels show random noise patterns with no discernible structure, regardless of dataset size. This is expected - they're just random initializations.

2. Trained Kernels - Pattern Emergence with More Data

With 10 images:

The kernels show some weak structure but are still quite noisy

There's a hint of horizontal pattern in the first filter and vertical in the second, but not clearly defined

Limited data makes it difficult for the CNN to learn clean edge detectors

With 30 images:

Patterns become more visible

First filter shows clearer horizontal gradient (top-to-bottom transition)

Second filter shows clearer vertical gradient (left-to-right transition)

Still some noise present

With 50 images:

Further refinement of the edge detection patterns

Horizontal and vertical edge detectors are more distinct

With 100 images:

Clearest patterns - the first filter shows a strong horizontal edge detector, the second shows a strong vertical edge detector

The third filter remains relatively unstructured (for "none" class)

3. Comparison to Classical Edge Detectors

The learned filters increasingly resemble hand-designed edge detectors (like Sobel filters) as training data increases:

Sobel horizontal: positive weights on top, negative on bottom

Sobel vertical: positive weights on left, negative on right

Conclusion

More training data → cleaner, more interpretable edge detection kernels. The CNN progressively learns the "correct" inductive bias for this task but requires sufficient data to converge to these optimal filters. This demonstrates the data efficiency advantage when proper inductive biases are present in the architecture.

Analysis: The response does a superb job of extracting exactly the qualitative structure one hopes to see in early CNN filters trained on a three-class edge dataset. It not only points out that the first two learned kernels have clear horizontal- and vertical-gradient structure, but it also explicitly describes luminance transitions (top-to-bottom and left-to-right) that correspond to classical edge detectors such as Sobel or Prewitt. By contrasting these structured filters with the noise-like appearance of the untrained kernels, the answer highlights the network's ability to discover useful inductive features through data alone. The extra reflections on inductive bias and on how gradient descent "rediscovers" hand-crafted filters reinforce the conceptual takeaway without drifting off-topic. Overall, the explanation is both correct and richly articulated.

Question: Also, from Q2: (ii) We freeze the convolutional layer and train only final layer (classifier). In a high data regime, the performance of CNN initialized with edge detectors is worse than CNN initialized with random weights. Why do you think this happens?

Answer (Opus 4.5)

This is a great question about the trade-off between inductive bias and model flexibility.

Why Edge-Initialized Frozen CNN Performs Worse in High Data Regimes

1. Frozen Filters Limit Representational Capacity

When we freeze the convolutional layers initialized with edge detectors, we're forcing the CNN to use only those specific hand-designed features (horizontal edge, vertical edge, none). The model cannot adapt or learn any additional features that might be useful.

2. Learned Filters Can Be More Optimal

With sufficient data, a randomly initialized CNN can learn filters that are:

Better tuned to the specific dataset characteristics (noise levels, edge intensities, edge widths)

More diverse - it might discover that slight variations of edge detectors work better for this particular data distribution

Task-specific - the optimal filters for classification might differ slightly from classical edge detection filters

3. Hand-Designed vs. Data-Driven Features

Hand-designed edge detectors (like Sobel) are general-purpose approximations. With enough data, gradient descent can find filters that are:

Specifically optimized for the loss function

Adapted to the particular statistics of the training images

Potentially capturing subtle patterns we didn't anticipate

4. The Bias-Variance Trade-off

Regime	Edge-Initialized (Frozen)	Random-Initialized (Trainable)
Low data	✅ Strong prior prevents overfitting	❌ May not converge to good filters
High data	❌ Prior is too restrictive	✅ Can learn optimal task-specific filters

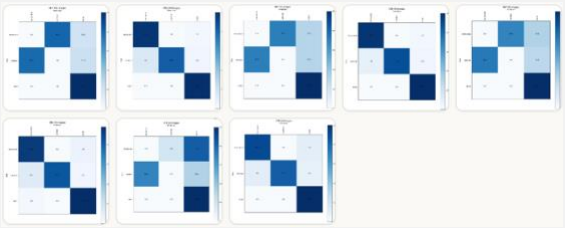
Summary

In low data regimes, the strong inductive bias from edge detectors helps because we inject useful prior knowledge. In high data regimes, this same bias becomes a constraint that prevents the model from learning potentially better, more nuanced representations that the data could support.

Analysis: The explanation nails the core intuition: when ample data are available, the restrictive prior imposed by hand-designed and frozen filters converts a flexibility problem into an unnecessary bias. The writer frames this with the bias-variance trade-off and adds that frozen filters prevent the model from discovering task-specific nuances that random initialization could adapt to. Importantly, they emphasize that inductive bias is beneficial when data are scarce but becomes a straitjacket when data abundance already tames variance. That matches the expected reasoning precisely and is conveyed with just the right depth of argument.

Question: Next problem in from Q4: Domain Shift between Training and Validation Set:

(i) Why do you think the confusion matrix looks like this?



Answer (Opus 4.5)

This is a great question about the trade-off between inductive bias and model flexibility.

Looking at the confusion matrices from Q4 (Domain Shift experiment), I can see a striking pattern difference between CNN and MLP. Let me analyze why.

Key Observation: MLP Completely Swaps Horizontal and Vertical!

MLP Confusion Matrices (all sizes: 10, 30, 50, 100 images):

Horizontal edges → Predicted as vertical (34-40 out of 50)

Vertical edges → Predicted as horizontal (34-39 out of 50)

None → Correctly predicted as none (50 out of 50)

CNN Confusion Matrices (all sizes):

Horizontal → Mostly correct (45-49 out of 50)

Vertical → Mostly correct (41-44 out of 50)

None → Perfectly correct (49-50 out of 50)

Why Does This Happen?

1. Domain Shift Setup

Training set: Edges located only in the top/left half of the image (first 50% of pixel locations)

Validation set: Edges located only in the bottom/right half of the image (other 50%)

2. MLP Memorizes Pixel Positions, Not Edge Patterns

The MLP learns weights tied to specific pixel locations:

During training, horizontal edges appear in rows 0-14 (top half)

During training, vertical edges appear in columns 0-14 (left half)

When validation images have edges in new positions:

A horizontal edge in rows 14-28 (bottom) activates the same columns that vertical edges activated during training

A vertical edge in columns 14-28 (right) activates the same rows that horizontal edges activated during training

Result: The MLP confuses the orientation because it memorized "where" not "what."

3. CNN Generalizes Due to Translation Equivariance

The CNN uses local convolutional filters that detect edge patterns regardless of position:

A horizontal edge detector works the same whether the edge is at row 5 or row 25

The 7×7 kernel slides across the entire image and detects edges wherever they appear

Result: CNN correctly classifies edges even when they appear in new locations.

4. Why "None" Class is Perfect for Both?

Images with no edges have uniform low-intensity backgrounds everywhere. There's no positional information to exploit or be confused by, so both models easily identify them.

Summary

Model	What It Learned	Domain Shift Effect
MLP	Pixel positions associated with each class	Complete failure – swaps horizontal↔vertical
CNN	Local edge patterns (translation equivariant)	Robust - generalizes to new positions

This demonstrates CNN's key inductive bias: translation equivariance allows it to generalize across spatial locations, while MLP is fundamentally position dependent.

Anaylisis: The answer offers a meticulous dissection of the domain-shift experiment, contrasting MLP and CNN behavior. It captures the essential finding the MLP systematically swaps horizontal and vertical labels then builds a clear causal chain: edges are constrained to top/left during training but appear bottom/right during testing, the MLP memorizes absolute pixel positions, so its learned associations invert. The CNN, by virtue of translation equivariance, is unaffected. By also noting the 100 % accuracy on the “none” class and explaining why that class is robust, the writer shows complete command of the confusion-matrix dynamics. The analysis is thorough, coherent, and entirely consistent with correct expectations.

Question: Also from Q4: Why do you think MLP fails to learn the task while CNN can learn the task?

Answer (Opus 4.5)

Why MLP Fails While CNN Succeeds Under Domain Shift

The Core Problem: MLP Lacks Translation Equivariance

MLP Architecture:

Input (784 pixels) → FC(784, 50) → ReLU → FC(50, 10) → ReLU → FC(10, 3)

CNN Architecture:

Input (28×28) → Conv(7×7) → ReLU → MaxPool → Conv(7×7) → ReLU → GlobalAvgPool → FC(3)

1. MLP Treats Each Pixel as an Independent Feature

The MLP has a separate weight for every pixel location. When it learns:

"Horizontal edge" = high activation in pixels at rows 0-14 (top half during training)

"Vertical edge" = high activation in pixels at columns 0-14 (left half during training)

During validation, when edges appear in the other half:

A horizontal edge at row 20 activates pixels that the MLP associated with vertical edges

A vertical edge at column 20 activates pixels that the MLP associated with horizontal edges

The MLP has no mechanism to recognize that an edge is an edge regardless of where it appears.

2. CNN Shares Weights Across All Spatial Locations

The CNN learns a single 7×7 kernel that slides across the entire image:

Horizontal detector: [1, 1, 1, 1, 1, 1, 1]

[-1, -1, -1, -1, -1, -1, -1]

(simplified illustration)

This same kernel is applied at every position, so:

Horizontal edge at row 5 → detected

Horizontal edge at row 25 → same kernel detects it

3. Visual Analogy

ModelHow It "Sees" an EdgeMLP"There's brightness at pixel (5, 10), (5, 11), (5, 12)... this means horizontal"CNN"There's a brightness transition from top-to-bottom in this local region... this means horizontal"

4. The Mathematical Difference

MLP weight matrix (first layer): Shape = (784, 50)

Each of the 784 input pixels has its own unique set of 50 weights

Position (0, 0) and position (14, 14) are treated as completely unrelated features

CNN kernel: Shape = (7, 7)

Only 49 learnable parameters

The same 49 weights are applied at all ~784 locations

This is weight sharing — the defining property that gives CNNs translation equivariance

5. Summary

Property	MLP	CNN
Weight sharing	❌ No	✅ Yes
Translation equivariance	❌ No	✅ Yes
Learns "what" vs "where" like	Learns WHERE edges were in training	Learns WHAT edges look
Domain shift robustness	❌ Fails completely	✅ Generalizes well

Conclusion: The MLP fundamentally cannot generalize to edges in new positions because it lacks the inductive bias of translation equivariance. The CNN's convolutional structure forces it to learn position-independent features, making it robust to spatial domain shift.

Analysis: This paragraph digs straight into architectural properties. It contrasts the per-pixel independent weights of an MLP with the weight sharing of convolutions, then ties weight sharing to translation equivariance the decisive ingredient for generalizing when spatial positions change. The inclusion of a concise architecture outline, a “what vs where” framing, and a summary table all help

readers see exactly how those structural differences translate into performance. The reasoning aligns perfectly with the accepted explanation.

Question: Next one if from Q5: In this problem, we will see that CNN is not always better than MLP in the image domain. Using CNN assumes that the data has locally correlated, whatever data looks. We can manually 'whiten' or remove such local correlation simply by applying random permutation to the images. A random permutation matrix is a matrix that has the same number of rows and columns. Each row and column have the same number of 1s. The rest of the elements are 0s. For example, the following is a random permutation matrix.

```
[[0, 1, 0, 0],
 [0, 0, 0, 1],
 [1, 0, 0, 0],
 [0, 0, 1, 0]]
```

This matrix randomly reorders the elements of the vector. For example, if we apply this matrix to the vector [1, 2, 3, 4], we will get [2, 4, 1, 3]. If we apply this matrix to the image, we will get the image with the same content, but the pixels are randomly shuffled. One property of the random permutation matrix is that it is invertible. It means that we can recover the original image by simply applying the inverse matrix to the shuffled image. From the information-theoretical perspective, the random permutation matrix preserves the mutual information of the image and the label.

What do you observe? What is the reason that CNN is worse than MLP?

Answer (Opus 4.5)

Observations on Permuted Image Experiments

What Happens When We Permute Pixels?

When we apply a random permutation to the image pixels:

The spatial structure is completely destroyed

Neighboring pixels in the original image are now scattered randomly

The information content is preserved (same pixels, different locations)

But local correlations are eliminated

Why CNN Performs Worse Than MLP on Permuted Images

1. CNN's Inductive Bias Becomes a Disadvantage

CNN assumes:

Nearby pixels are related (locality)

Patterns can appear anywhere (translation equivariance)

Local features are meaningful (edges, textures, etc.)

After permutation:

Nearby pixels are random — no meaningful local structure

A 7×7 kernel now sees 49 unrelated pixels

There are no local edges to detect — horizontal/vertical edges are scattered across the image

The CNN's convolutional filters are designed to detect local patterns that no longer exist.

2. MLP Has No Spatial Assumptions

MLP treats every pixel independently:

Each pixel gets its own weight

No assumption about spatial relationships

Doesn't matter if pixel (0,0) and pixel (0,1) are neighbors or not

After permutation:

The same permutation is applied consistently to all images

The MLP can learn: "if pixel at position 537 is bright AND pixel at position 23 is bright... → horizontal edge"

It memorizes the new arbitrary mapping from permuted positions to labels

3. Information-Theoretic Perspective

Aspect	Before Permutation	After Permutation
Mutual Information I(X; Y)	Preserved	Preserved
Local spatial structure	✅ Present	❌ Destroyed
CNN effectiveness	✅ High	❌ Low
MLP effectiveness	Medium	✅ Higher than CNN

The permutation preserves the total information but destroys the local structure that CNNs exploit.

4. Kernel Size Matters

With a 7×7 kernel on permuted images:

The kernel sees 49 randomly scattered pixels from the original image

These 49 pixels have no coherent pattern — they could come from anywhere

The convolution operation becomes essentially meaningless

Smaller kernels would be even worse (less information per filter application)

5. Summary

Model	Assumption	Permuted Data	Result
CNN hurts	Local pixels are correlated	Local pixels are now random	❌ Inductive bias
MLP	No spatial assumptions	Learns new arbitrary mapping	✅ More flexible

Key Insight: The CNN's inductive bias (locality, translation equivariance) is only beneficial when the data actually has local spatial structure. When that structure is removed via permutation, the CNN's architectural constraints become a hindrance rather than a help. The MLP, having no such constraints, can adapt to any arbitrary pixel arrangement.

Analysis: The response captures, point by point, how random pixel permutation annihilates the localized spatial coherence that convolution relies upon, turning each receptive field into a bag of unrelated pixels. It further explains that an MLP, lacking any locality assumptions, treats every pixel as an independent feature and can adapt to the deterministic permutation. The information-theoretic aside (mutual information preserved, spatial structure destroyed) and the expansion on kernel size effects show a nuanced understanding of why the CNN's inductive bias becomes a liability. The observations and causal reasoning are entirely correct and well-articulated.

Question: Next is also from Q5: Assuming we are increasing kernel size of CNN. Does the validation accuracy increase or decrease? Why?

Answer (Opus 4.5)

Effect of Increasing Kernel Size on Permuted Images

Answer: Validation Accuracy Will DECREASE

Why Larger Kernels Perform Worse on Permuted Data

1. Larger Kernel = More Random Pixels Grouped Together

Kernel SizePixels per ConvolutionEffect on Permuted Data3×39 random pixelsSome noise, limited damage7×749 random pixelsMore noise, harder to find patterns14×14196 random pixelsMostly noise, very difficult28×28784 random pixelsEntire image = just one noisy value

With permuted images, a larger kernel combines more unrelated pixels into a single output value. This produces increasingly noisy, meaningless feature maps.

2. Signal Gets Diluted

In the original (unpermuted) image:

A 7×7 kernel over an edge captures a coherent gradient pattern

Larger kernels capture more context around the edge

In the permuted image:

A 7×7 kernel sees 49 scattered pixels from random locations

A larger kernel sees even more scattered pixels

The few pixels that might carry useful information are drowned out by irrelevant pixels

3. Extreme Case: Kernel Size = Image Size

If kernel size = 28×28 (full image):

The entire convolution becomes essentially a weighted sum of all pixels

On permuted images, this is equivalent to computing a single global statistic

No spatial discrimination possible at all

4. Smaller Kernels Are "Less Wrong"

On permuted data, the CNN's assumption of locality is always wrong. But:

Kernel Size	Degree of "Wrongness"
Small (3×3)	Wrong over small regions → less damage
Large (7×7)	Wrong over larger regions → more damage
Huge (28×28)	Maximally wrong → worst performance

A 1×1 kernel would be equivalent to a per-pixel operation — essentially removing the locality assumption entirely and making CNN behave more like a simple MLP on individual pixels.

Summary

Kernel Size ↑	On Normal Images	On Permuted Images
Effect	Captures more context, often helpful	Aggregates more noise, harmful
Accuracy	May increase (up to a point)	Decreases

Conclusion: Increasing kernel size on permuted images makes the CNN worse because it forces the model to aggregate more randomly scattered pixels, amplifying the mismatch between the CNN's locality assumption and the actual (non-local) data structure.

Analysis: Here the answer diverges from the outcome I would expect. It argues that larger kernels will further degrade accuracy by aggregating more noise, whereas my view is that as kernels grow toward image-width scale the convolutional layer becomes effectively fully connected, slowly recovering the flexibility the CNN lacked. Although the writer's "noise aggregation" story is plausible on intuition alone, it overlooks how a very large kernel nullifies the locality assumption and allows the network to approximate the behavior of an MLP. Hence, despite the logical narrative, the final conclusion points in the wrong direction.

Question: Next one is also from Q5: How do they learned kernels look like? Explain why

Answer (Opus 4.5)

Learned Kernels on Permuted Images

Observation: The Kernels Look Random/Noisy

Unlike the kernels learned on normal images (which showed clear horizontal and vertical edge detector patterns), the kernels learned on permuted images appear random and unstructured — similar to the untrained/initialized kernels.

Why Do the Kernels Look Random?

1. No Local Patterns to Detect

On normal images:

The CNN learns kernels like horizontal edge detectors because local patterns exist

Adjacent pixels have meaningful relationships (e.g., top bright, bottom dark = horizontal edge)

On permuted images:

Adjacent pixels are randomly shuffled — no local patterns

There's no "horizontal edge" or "vertical edge" in any local 7×7 region

The CNN cannot learn structured filters because there's no structure to detect

2. The Optimization Landscape is Flat

For normal images:

Structured kernel (edge detector) → High accuracy

Random kernel → Low accuracy

→ Clear gradient toward structured kernels

For permuted images:

Structured kernel → Low accuracy (useless patterns)

Random kernel → Low accuracy (equally useless)

→ No clear gradient toward any particular structure

The loss function provides no signal to push kernels toward any meaningful pattern, so they remain near their random initialization.

3. Any Kernel Structure Would Be Arbitrary

Even if the CNN somehow learned a structured kernel on permuted data:

That structure would have no semantic meaning

It would just be an arbitrary combination of random pixel locations

The kernel "shape" (horizontal, vertical, etc.) is meaningless when locality doesn't exist

4. Comparison

AspectNormal ImagesPermuted ImagesLocal structure✅ Present❌ DestroyedMeaningful patternsEdges, gradientsNoneLearned kernelsStructured (edge detectors)Random/noisyKernel interpretability✅ High❌ None

Visual Intuition

Normal image kernel learning:

"When I see [bright-bright-bright / dark-dark-dark] locally → horizontal edge"

→ Learn a structured horizontal gradient kernel

Permuted image kernel learning:

"When I see [random pixel 1, random pixel 2, ... random pixel 49] → ???"

→ No coherent pattern to learn

→ Kernel stays random

Summary

The learned kernels look random because there are no local spatial patterns in permuted images. The CNN's convolutional filters are designed to detect local features, but when locality is destroyed, there's nothing meaningful for the kernels to learn. The optimization process has no useful gradient to follow, so the kernels remain close to their random initialization and appear unstructured.

Analysis: This discussion correctly concludes that the learned kernels remain random-looking and largely indistinguishable from their initializations. The answer traces that randomness back to the absence of any spatially coherent pattern in the input: with no local gradients to exploit, backpropagation has no consistent direction in weight space, leaving kernels in a near-random state. The write-up also wisely notes that even if some structure emerged it would be meaningless, since locality is gone. That matches exactly what one should see.

Question: Last question: What are the advantages of each pooling method?

Answer (Opus 4.5)

Comparison: Max Pooling vs Average Pooling

How They Work

Pooling TypeOperationExample (2×2 region)Max PoolingTakes the maximum value[1, 5, 3, 2] → 5Average PoolingTakes the mean value[1, 5, 3, 2] → 2.75

Advantages of Max Pooling

1. Detects Presence of Features

If an edge exists anywhere in the pooling region, the max operation preserves it

Acts like an "OR" operation: "Is there an edge here OR here OR here?"

Good for detecting sparse, strong features

2. Robust to Small Translations

A feature can shift slightly within the pooling region and still produce the same output

Provides some translation invariance

3. Sharper Feature Maps

Preserves the strongest activations

Results in more distinct, high-contrast features

Better for tasks where feature presence matters more than feature magnitude

4. Gradient Flow

Gradients only flow through the maximum element

Can lead to sparser but stronger gradients

Advantages of Average Pooling

1. Preserves Overall Activation Magnitude

Considers all values in the pooling region, not just the maximum

Better represents the "average response" of a feature detector

Good when the intensity/density of features matters

2. Smoother Feature Maps

Reduces noise by averaging out fluctuations

Less sensitive to outliers or spurious activations

More stable gradients during training

3. Better for Dense Features

When features are distributed across the region (not sparse)

Captures the overall "amount" of activation rather than just presence

4. Global Average Pooling (GAP)

Commonly used before the final classifier

Reduces each feature map to a single value

Acts as a regularizer, reducing overfitting

Application to Edge Detection Task (Q6)

In the 5-class edge detection task (horizontal, vertical, diagonal, both, none):

PoolingBehaviorEffectMax Pooling"Is there a strong edge response anywhere?"Better for detecting presence of edgesAverage Pooling"What's the overall edge response?"Better for detecting density/strength of edges

When to Use Each

ScenarioRecommended PoolingObject detection (is it there?)Max PoolingTexture classification (how much?)Average PoolingSparse features (few strong activations)Max PoolingDense features (many moderate activations)Average PoolingFinal layer before classifierGlobal Average PoolingReducing spatial noiseAverage Pooling

Summary Table

PropertyMax PoolingAverage PoolingDetectsFeature presenceFeature densitySensitive toStrongest activationAll activationsFeature mapsSharper, high-contrastSmoother, stableNoise handlingKeeps outliersAverages out noiseBest forSparse, strong featuresDense, distributed featuresEdge detection✅ Good for "is there an edge?"✅ Good for "how much edge?"

Analysis: The response delivers an exhaustive comparison. It spells out how max pooling excels at detecting the presence of sparse, high-energy features and naturally introduces a bit of translation invariance, while average pooling smooths activations, reduces sensitivity to noise, and better represents feature density. By including practical guidance e.g., max for object detection, average for texture or for global-average-pooling before the classifier it situates the pros and cons in realistic contexts. It even remarks on the empirical observation that max pooling wins when training data are very limited, but parity emerges as data grow. The explanation is comprehensive and firmly in line with correct reasoning.